

Betriebssysteme

Speicherverwaltung

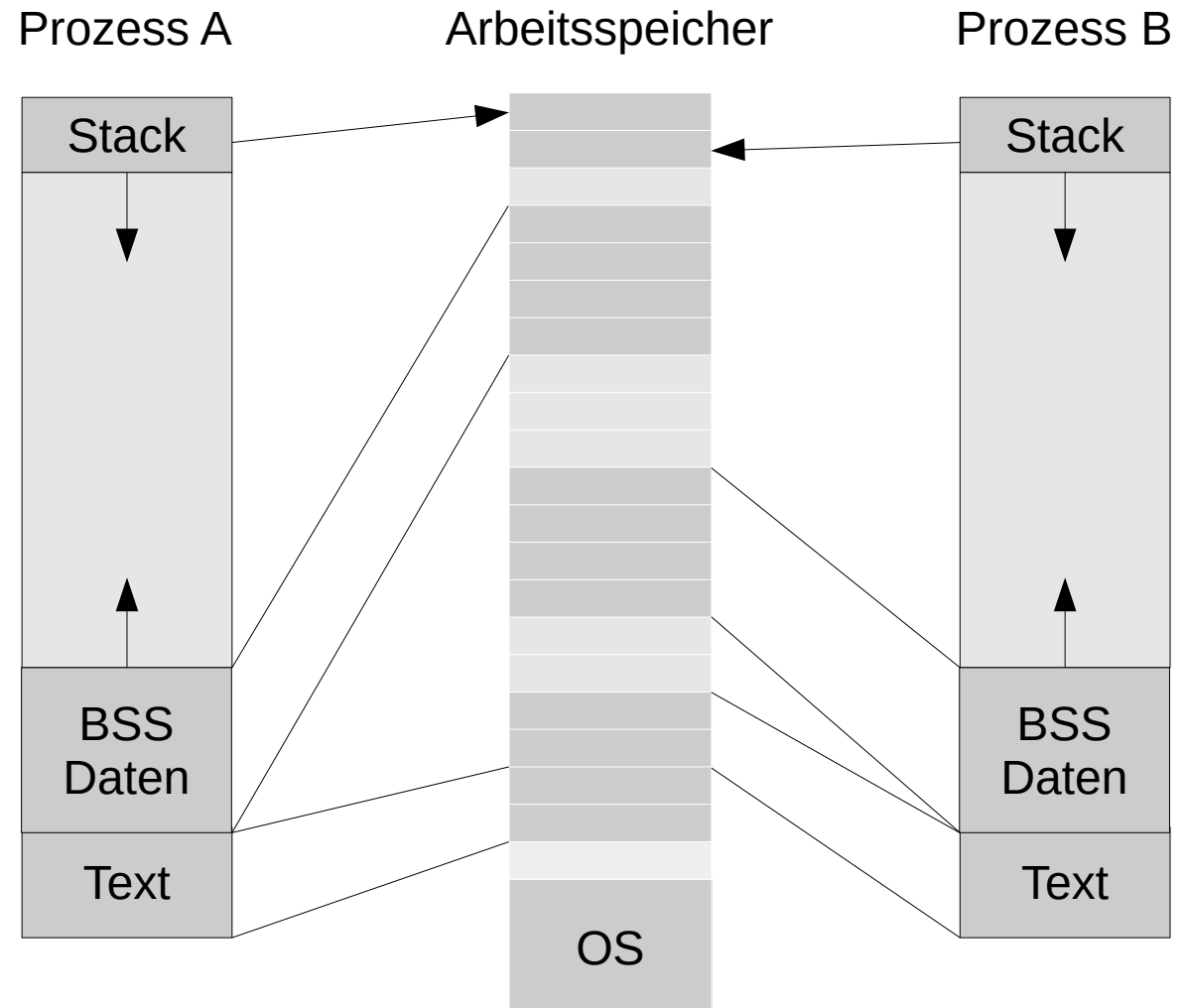
Andreas Scheibenpflug, MSc
SE Bachelor (BB/VZ), 2. Semester

Speicherverwaltung

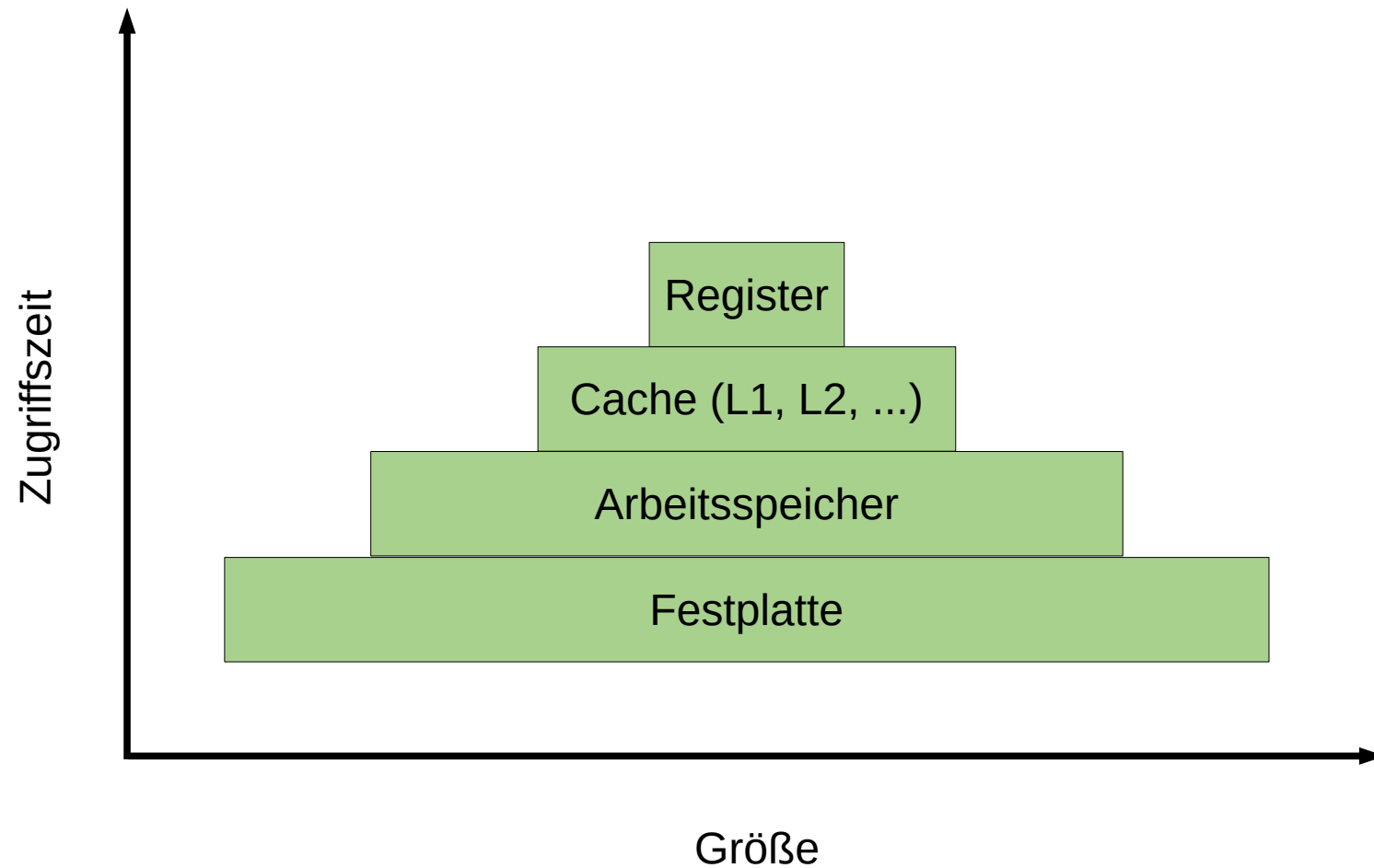
- Prozesse benötigen Arbeitsspeicher für
 - Code (Instruktionen)
 - Daten (Variablen, Heap, Stack)
- OS verwaltet den Hauptspeicher (RAM)
 - Zuteilung von Speicher zu Prozessen
 - Allokieren / Freigeben von Speicher
 - Auslagern (Paging) auf die Festplatte
 - Optimale Ausnutzung des Speichers

Speicherlayout (Linux)

- Text Segment
 - Instruktionen/Code
- Daten
 - Initialisierte Daten, z.B. statische Variablen, globale Variablen
- BSS
 - „Block Started by Symbol“
 - Nicht initialisierte Variablen
 - Heap
- Stack
 - Lokale Variablen, Rücksprungadressen, Argumente
 - Stack Pointer zeigt auf die Startadresse dieses Bereichs



Speicherhierarchien



Abstraktion

- Ein OS stellt auch mit der Speicherverwaltung eine Abstraktionsschicht für Programme im User Space zur Verfügung
- Virtual Memory ist dabei die gebräuchlichste Technik
- Wir sehen uns zur Einleitung aber auch noch weitere Möglichkeiten an
 - Keine Abstraktion
 - Adressräume

Keine Abstraktion

- Fand Anwendung bei z.B. Mainframe Computer vor 1960 oder PCs vor 1980
- Findet noch Anwendung bei manchen Eingebetteten Systemen
- Jede Operation in Bezug auf Arbeitsspeicher, die eine Anwendung durchführt, wird direkt 1:1 auf dem physischen Speicher ausgeführt
 - z.B. JMP 28 springt an die tatsächliche Speicheradresse 28
- Konsequenz: Es kann nur ein Prozess im Speicher gehalten werden da
 - Speicherzugriffsverletzungen (Segfault) nicht festgestellt bzw. verhindert werden können
 - Adressierung bei mehreren Prozessen im Speicher nicht mehr funktionieren würde
- Mögliche Lösungsansätze
 - Anpassung von Adressen relativ zur Startspeicheradresse
 - Auslagern des Speicherabbilds von inaktiven Prozessen auf die Festplatte

Keine Abstraktion - Beispiel



Prozesse	Speicheradressen
Prozess 2	
JMP 8	1016
	1012
	1008
	1004
	1000
	...
	...
Prozess 1	
	20
JMP 8	16
	12
	8
	4
	0

Adressräume

- Jeder Prozess hat seinen eigenen Adressraum
 - d.h. jedem Prozess wird eine Menge an Adressen zugewiesen, die dieser verwenden darf
- Dynamic Relocation
 - Base Register ist die Startadresse des Programms
 - Limit Register ist die Programmlänge
 - Greift das Programm auf Speicheradressen zu, wird die Adresse mit dem Base Register addiert und überprüft, ob die Endadresse des Programms nicht überschritten wird
 - Nachteil: Zusätzliche Operationen (Addition + Vergleich) bei jedem Speicherzugriff

Dynamic Relocation - Beispiel

Prozesse	Speicheradressen
Prozess 2	
JMP 8	1016
	1012
	1008
	1004
	1000
	...
	...
Prozess 1	
	20
JMP 8	16
	12
	8
	4
	0

1000 + 8 &&
1008 <= 1016
→ JMP 1008

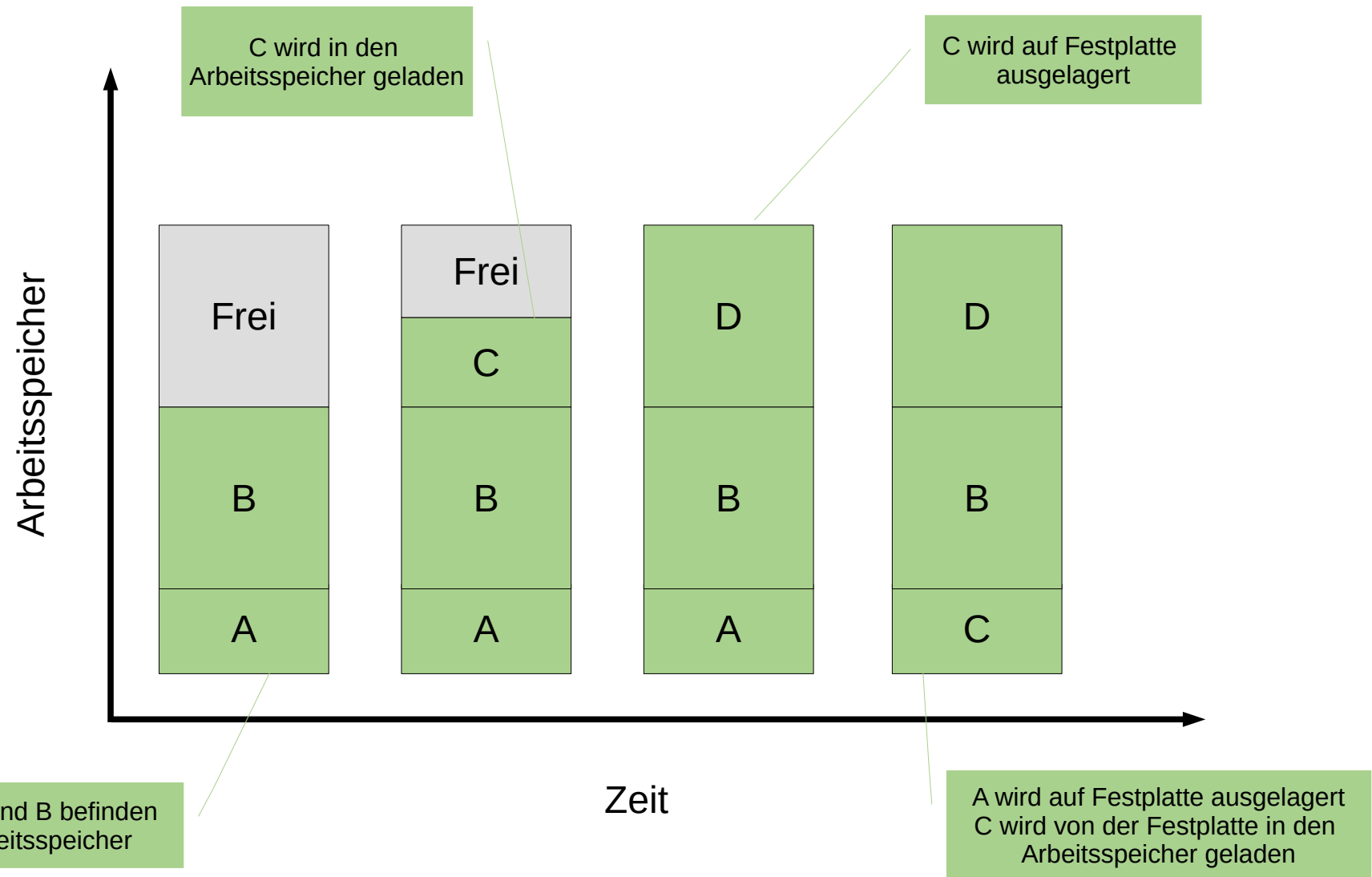
Limit Register

Base Register

Swapping

- Es ist unter Umständen nicht möglich, alle Prozesse im Arbeitsspeicher zu halten
 - Wenn viele Prozesse gleichzeitig laufen und/oder Prozesse große Mengen an Arbeitsspeicher benötigen
- Swapping bedeutet,
 - den Speicher des kompletten Prozesses auf die Festplatte auszulagern, wenn dieser nicht läuft
 - den Speicher des kompletten Prozesses wieder von der Festplatte in den Arbeitsspeicher zu laden, wenn der Prozess fortgesetzt wird
- Damit wird Arbeitsspeicher für andere laufende Prozesse frei

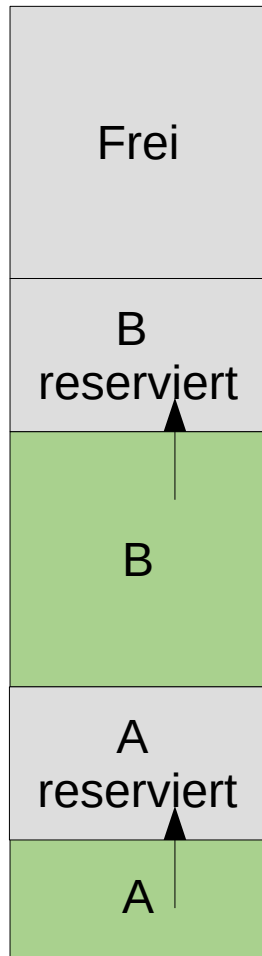
Swapping - Beispiel



malloc anyone?

- Bisher haben wir einem Programm eine fixe Größe im Arbeitsspeicher zugewiesen
- Es muss aber auch dynamische Speicherallokierung beachtet werden
 - Hat einen Einfluss auf die Aufteilung des Arbeitsspeichers
 - Anders gesagt: Das Datensegment eines Prozesses kann zur Laufzeit wachsen und schrumpfen
- Wenn also ein Prozess über seine Grenzen hinauswachsen würde, muss er im Speicher umgesiedelt werden
 - Schlecht für die Performance
 - Daher ist es eine bessere Idee, Lücken zwischen den Speicherbereichen von Prozessen zu schaffen
 - Beim Swappen wird aber nur der tatsächlich belegte Teil des Speichers auf die Festplatte geschrieben

Swapping mit Lücken



- Wenn reservierter Speicher ausgeht:
 - Relocation in einen Speicherbereich der groß genug ist
 - Swapping und Pausieren, bis dass ein größerer Speicherbereich frei ist
 - Beenden (Out Of Memory Exception)

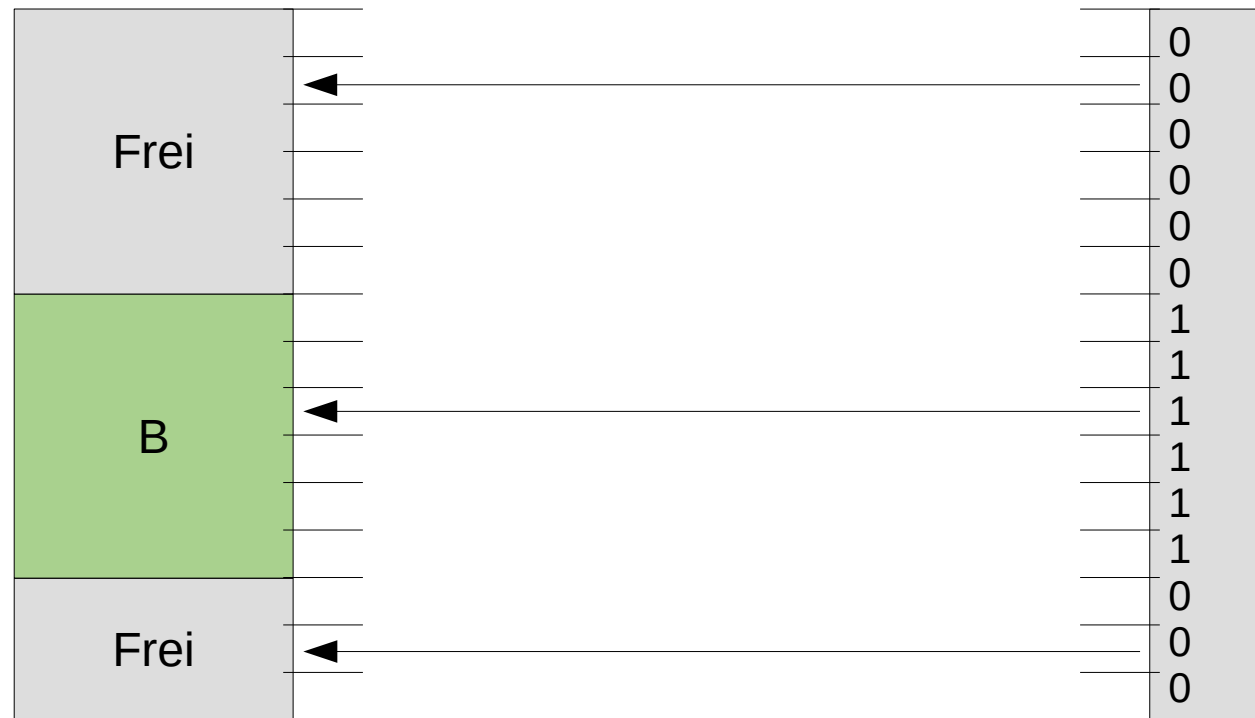
Keeping Track of Memory

- Wie wir im Swapping Beispiel gesehen haben, trifft das OS Entscheidungen, an welche Stellen im Arbeitsspeicher Programme geladen werden
- Voraussetzung dafür ist, dass das OS weiß,
 - wo wie viel Speicher frei ist
 - wie viel Speicher als Buffer (Lücke) freigehalten werden sollte
- Wir sehen uns im Folgenden zwei Strategien zum Verwalten der Speicherbelegung an
 - Bitmaps
 - Verkettete Listen

Bitmaps

- Jedes Bit in einer Bitmap ist einer Allocation Unit zugeordnet
 - Allocation Unit: Adressierbare Speichereinheit
 - 0 bedeutet Allocation Unit ist frei
 - 1 bedeutet Allocation Unit ist belegt
- Größe einer Allocation Unit hat einen Einfluss auf die Größe der Bitmap
 - Umso kleiner die Allocation Unit, umso größer wird die Bitmap
 - Bsp: AU 4 Bytes, d.h. 1 Bit pro 4 Bytes, bei 2 GB RAM würde also die Bitmap 64 MB groß sein

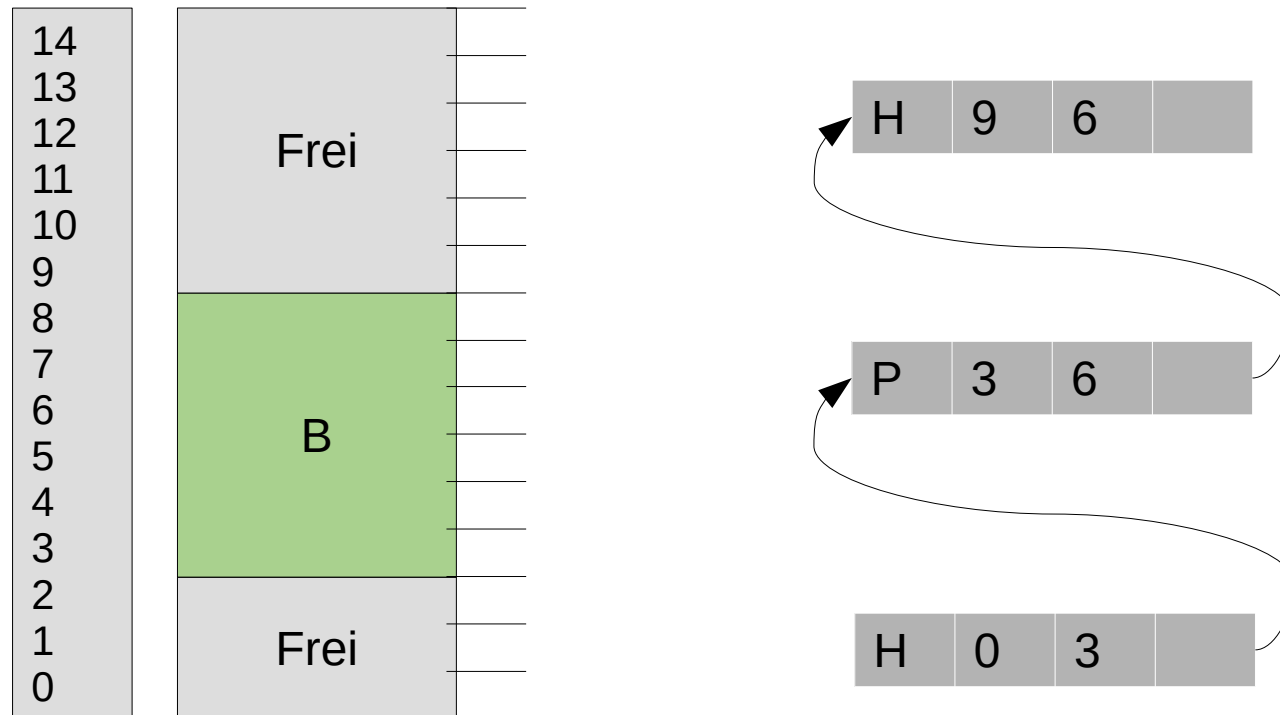
Bitmaps - Beispiel



Verkettete Listen

- Anstelle einer Bitmap wird eine verkettete Liste mit 2 Knotentypen (Process, Hole) verwendet
- Knoten speichern
 - Typ
 - P – Process: Speicherbelegung eines Prozesses
 - H – Hole: Block aus freiem Speicher
 - Startadresse
 - Länge
- Länge der Liste und der Speicherbedarf der Liste hängt damit von der Anzahl an Prozessen und freien Speicherblöcken ab
 - Im Gegensatz zur Bitmap, die eine konstante Größe (abhängig von der Größe des Arbeitsspeichers) hat

Verkettete Listen - Beispiel



Verkettete Listen - Varianten

- Wird ein neuer Prozess in den Speicher geladen, muss entschieden werden, welcher freie Platz verwendet werden soll
 - First Fit: Durchlaufen der Liste und Verwenden des ersten passenden Platzes
 - Best Fit: Durchlauf der Liste und Verwenden des kleinsten freien Platzes
 - Separate Listen für Prozesse und Holes als Optimierung
- Jede Variante hat Auswirkung auf Performance und die Menge an verschwendetem Arbeitsspeicher
 - Best Fit tendiert z.B. überraschender Weise dazu, kleine Speicherlöcher zu produzieren
 - Best Fit ist langsamer beim Speicherzuweisen als First Fit
 - First Fit tendiert zu größeren Speicherlücken

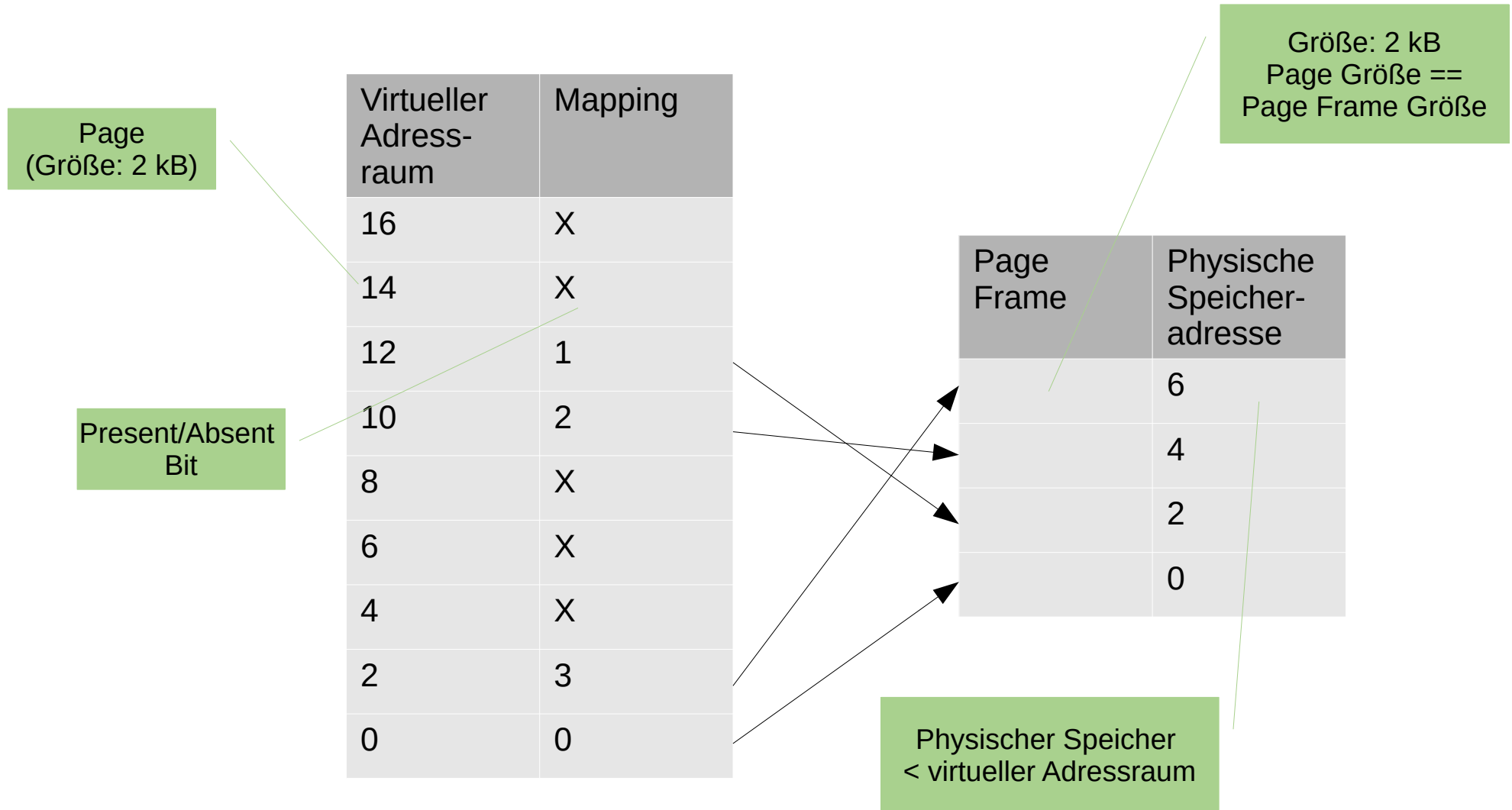
Virtual Memory

- Jeder Prozess hat seinen eigenen Adressraum
- Der Adressraum ist unterteilt in Seiten (Pages)
 - Eine Page ist eine aufsteigende Folge von Adressen
 - Pages werden auf den physischen Speicher gemappt
- Während ein Prozess ausgeführt wird, müssen sich nicht alle Pages des Prozesses im Hauptspeicher befinden
- Hardware führt das Adressmapping durch, wenn sich eine referenzierte Speicheradresse schon im physischen Speicher befindet (siehe Base/Limit Register)
- Befindet sich die referenzierte Speicheradresse nicht im Hauptspeicher, wird das OS benachrichtigt (Page Fault, wird vom Prozessor (MMU) ausgelöst) und lädt die Page, die die Adresse enthält
- Vorteil: Es kann eine größere Anzahl an Prozessen gleichzeitig im Speicher gehalten werden
 - Ist z.B. das Laden einer Page für einen Prozess notwendig, kann in der Zwischenzeit ein anderer, wartender Prozess ausgeführt werden

Virtual Memory

- Programme referenzieren virtuelle Adressen (vom Compiler generiert) in einem virtuellen Adressraum
- Die MMU (Memory Management Unit, Teil des Prozessors)
 - Mappt die virtuelle Adresse auf die physische Adresse
 - Stellt fest, ob die Page, die eine virtuelle Adresse enthält, sich im Hauptspeicher befindet (ob die Page einen korrespondierenden Page Frame hat)
 - Löst einen Page Fault aus, wenn sich die Adresse nicht im Hauptspeicher befindet
- Paging ist
 - das Laden von benötigten Pages von der Festplatte in den Arbeitsspeicher
 - das Auslagern von nicht benötigten Pages aus dem Arbeitsspeicher auf die Festplatte
 - Aufgabe des OS ist es, dass
 - benötigte Pages nachgeladen (eingelagert) werden
 - Page Frames auf die Festplatte ausgelagert werden, um Arbeitsspeicher freizugeben

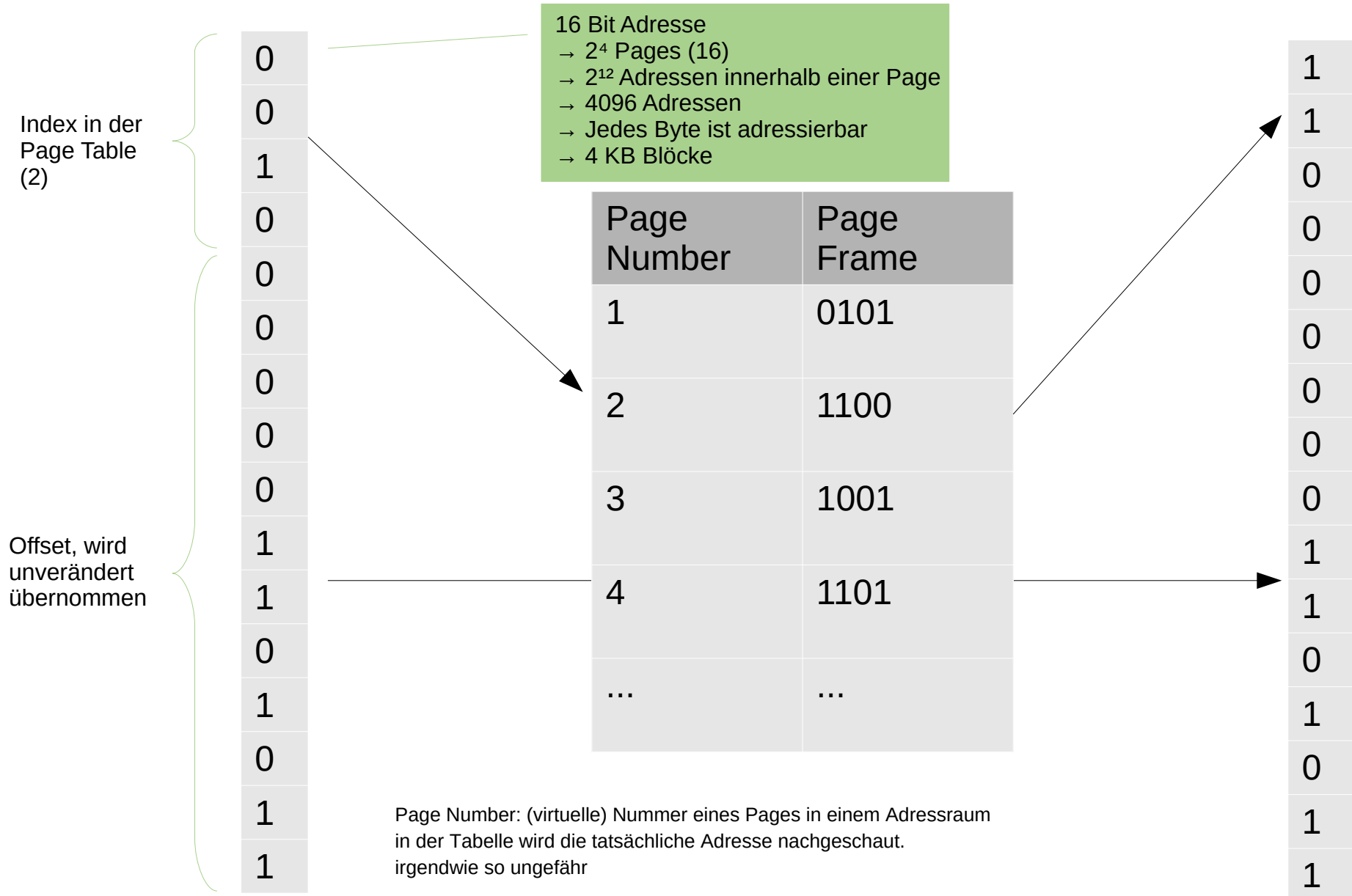
Virtual Memory - Beispiel



Page Table

- Page Table enthält
 - das Mapping zwischen Page und Page Frame (virtuellen und physischen Adresse)
 - Zusatzinformationen (z.B. Present/Absent Bit,...)
- Adresse besteht aus 2 Teilen
 - Page Number: Wird in der Page Table als Index für den Eintrag der Page Frame Nummer verwendet
 - Offset: Für die Adressierung innerhalb des Frames, wird unverändert übernommen

MMU – Vereinfachtes Beispiel



Page Table Einträge

- Tatsächlich enthält ein Eintrag in der Page Table mehr Information
 - Page Frame Number
 - Present/Absent Bit:
 - 0: Page ist nicht gemappt → Page Fault
 - 1: Page ist gemappt und es kann zugegriffen werden
 - Protection: Lese- und oder Schreibzugriff erlaubt
 - Modified: Wurde in die Page geschrieben, wird das Modified Bit gesetzt. Wurde die Page verändert, muss sie beim Freigeben auf die Platte zurückgeschrieben werden. Andernfalls ist keine Aktion des OS nötig
 - Referenced: Wird gesetzt, wenn eine Adresse von einer Aktion referenziert wird. Damit weiß das OS, dass diese Page ein schlechter Kandidat zum Freigeben ist, da sie noch, oder bald, verwendet wird
- Aber keine Information zu z.B. Speicherort auf der Festplatte, da dies Aufgabe des OS ist

Performance Aspekte

- Auflösen von virtuellen (auf physische) Adressen muss schnell sein
 - Wird bei jedem Speicherzugriff benötigt
 - Sollte daher HW-unterstützt werden, da schnell
- Virtueller Adressraum ist groß
 - Die Page Tables können sehr groß werden
 - Mit 32 Bit können ~1 Million Pages (2^{20}) adressiert werden
 - Mit 64 Bit können sehr sehr viele Pages adressiert werden
 - Nicht wirtschaftlich, HW Register für komplette Page Tables anzubieten
- Lösung ist ein Kompromiss: Translation Lookaside Buffer

cached lookups auf die page table -- wenn man wieder die selbe Adresse braucht, kann man gleich wieder drauf zugreifen, ohne den Umweg auf den RAM

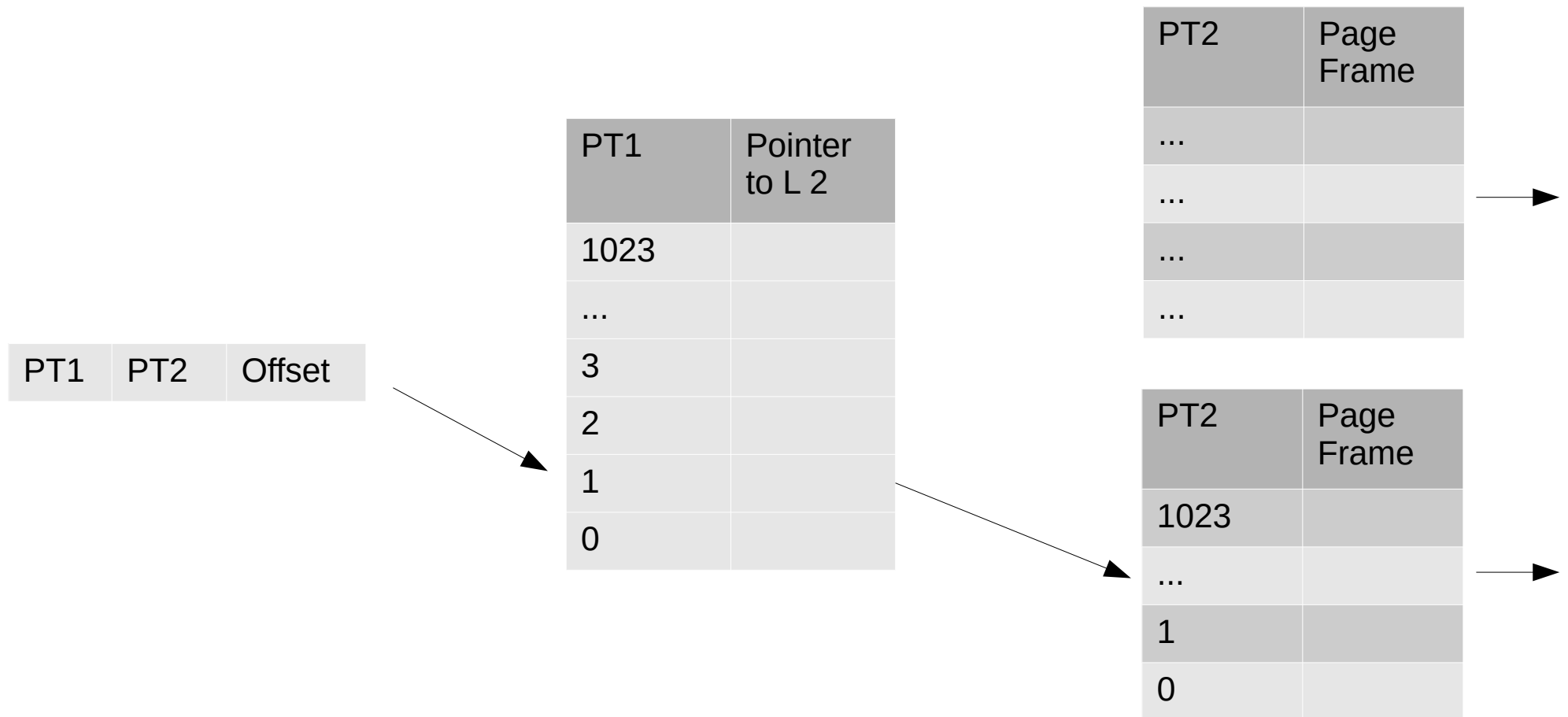
Translation Lookaside Buffer

- Technik zur Beschleunigung von Page Lookups
- Page Table ist im Arbeitsspeicher, TLB ist Teil der MMU und in HW implementiert
- Idee: Die meisten Programme greifen nur auf eine kleine Anzahl an Pages oft zu
- TLB ist Teil der MMU und ist eine kleine Mapping Tabelle, die den Umweg über die Page Table und damit den Arbeitsspeicher vermeidet
- Wird beim Anfordern einer Page das Mapping nicht im TLB gefunden, wird der Eintrag aus der Page Table geladen und der TLB aktualisiert

Multilevel Page Tables

- Technik für große virtuelle Adressräume
- Es müssen damit nicht immer alle Page Tables im Speicher gehalten werden
 - z.B. jene Page Tables für Bufferblöcke eines Programms, die für wachsenden Speicher reserviert sind, müssen nicht im Speicher gehalten werden
- Beispiel 32 Bit Adressen
 - Ersten 10 Bit für die Level 1 Page Table
 - Referenziert 4 MB Blöcke (→ 4 GB adressierbarer Speicher in 32 Bit Architekturen)
 - Nächsten 10 Bit für die Level 2 Page Table
 - Verbleibenden 12 Bit für den Offset
- z.B.
 - x86 (32 Bit): $2^{10} \times 2^{10} \times 2^{12} \rightarrow 4294967296 \text{ Bytes} \rightarrow 4 \text{ GB}$
 - x68_64, 4 Levels à 512 Byte $\rightarrow 2^9 \times 2^9 \times 2^9 \times 2^9 \times 2^{12} \rightarrow 2^{48} \rightarrow 256 \text{ TB}$

Multilevel Page Tables



Inverted Page Tables

- „Inverted“, weil die Tabelle ein Mapping von Frame auf Page ist
 - Eine Zeile der Tabelle entspricht einem Frame des Arbeitsspeichers und der zugeordneten Page
- Damit ist die Größe der Tabelle konstant
- Problem: Suche nach virtuellen Adressen ist langsam
- Lösung
 - Verwendung des TLB
 - Hashing der virtuellen Adressen um Suchen zu beschleunigen
- z.B. Itanium, UltraSPARC, PowerPC

Ersetzungsstrategien

- Wird eine Page benötigt, die sich nicht im Arbeitsspeicher befindet
 - Muss dass OS diese von der Festplatte laden
 - Eventuell eine Page aus dem Speicher entfernen
 - Die entfernte Page eventuell auf die Festplatte schreiben, wenn diese verändert wurde
 - Oder keine Aktion durchführen, wenn der Eintrag auf der Festplatte aktuell ist weil die Page nicht verändert wurde
- Der Page Replacement Algorithmus führt die Auswahl einer geeigneten Page zum Ersetzen mit der nachzuladenden Page durch
 - Im Optimalfall wird jene Page, die zeitlich möglichst weit entfernt verwendet wird, ersetzt
 - Das Problem ist, dass diese Information nicht verfügbar ist

Not recently used (NRU)

- Voraussetzung: Zu jeder Page in der Page Table wird folgende Information gespeichert
 - Referenced (R Bit): Wird gesetzt, wenn eine Page gelesen oder verändert wird
 - Modified (M Bit): Wird gesetzt, wenn eine Page verändert wird
- Wird im Normalfall von der Hardware aktuell gehalten (auch Verwaltung durch OS möglich)
- Zu bestimmten Zeitpunkten (z.B. Timer Interrupt) wird das R Bit zurückgesetzt (wird auch zurückgesetzt, aber vielleicht nicht mit timer)
 - Länger nicht verwendete Pages werden damit als nicht referenziert markiert

Not recently used (NRU)

- Daraus ergeben sich folgende Klassen
 - Klasse 0: nicht referenziert, nicht modifiziert
 - Klasse 1: nicht referenziert, modifiziert
 - Klasse 2: referenziert, nicht modifiziert
 - Klasse 3: referenziert, modifiziert wird wahrscheinlich wieder gebraucht
- NRU entfernt zufällig Seiten, beginnend mit der niedrigsten, nicht leeren Klasse
 - Besser nicht referenzierte Seiten zu entfernen, auch wenn diese modifiziert sind
- Vorteil: Einfach zu implementieren, für viele Fälle ausreichend bis hierher

FIFO / Second Chance

- Pages werden in eine verkettete Liste eingefügt
 - Bei einem Seitenfehler wird der älteste Eintrag einfach entfernt
 - Nachteil: Eintrag könnte noch benötigt werden → untauglich in der Praxis
- Second Chance Algorithmus
 - Es wird zusätzlich das R Bit verwendet, um zu überprüfen, ob die Seite noch referenziert ist
 - Ist das der Fall, wird die Page übersprungen, das R Bit gelöscht und die Page an das Ende der Liste gestellt
 - Danach wird der nächsten Eintrag in der FIFO Liste überprüft
 - Sind alle R Bits gesetzt, wird aus Second Chance FIFO und der älteste Eintrag entfernt
 - Vorteil: Vernünftiger Algorithmus, praktische Implementierungen vermeiden Umhängen von Knoten in der Liste aus Performance Gründen (Clock Algorithmus)

Least Recently Used (LRU)

- Idee: Eine Seite, die in letzter Zeit häufig benutzt wurde, wird wahrscheinlich auch in Zukunft häufig benutzt
- Benötigt wird eine Liste aus allen Pages, geordnet nach Benutzungshäufigkeit
- Problem dabei: Aufwändig zu implementieren bzw. langsam
 - Liste müsste bei jedem Speicherzugriff aktualisiert werden
 - Benötigt für jede Page einen Zähler (Counter) und dieser muss auch bei jedem Zugriff aktualisiert werden
- Es würde also zumindest HW Unterstützung benötigt werden, um dies effizient umsetzen zu können → selbst dann nicht praxistauglich
- Daher Annäherung über Algorithmus Not Frequently Used (NFU)

Not Frequently Used (NFU)

- Für jede Page gibt es einen (Software) Zähler
 - Ist zu Beginn 0
 - Bei einem Timer Interrupt werden alle Seiten durchlaufen und bei jenen, die das R Bit gesetzt haben, der Zähler inkrementiert
 - Die Seite mit dem niedrigsten Zähler wird zur Ersetzung ausgewählt
- Nebeneffekt: Wurde auf eine Seite oft zugegriffen, wird diese unter Umständen sehr lange im Speicher gehalten obwohl sie nicht mehr benötigt wird → zeitliche Komponente fehlt
- Erweiterung
 - Zähler werden 1 Bit nach rechts geschoben
 - Inkrementieren erfolgt durch Addieren des R Bits zum höchstwertigen (linken) Bit des Zählers
 - Wird auch als „Aging“ bezeichnet
- Dadurch „vergessen“ Zähler ihre Werte nach einer bestimmten Zeit

Aging Beispiel

Zeit	R-Bit Page 0	R-Bit Page 1	Page 0	Page 1
0	1	0	10000000	00000100
1	1	0	11000000	00000010
2	1	0	11100000	00000001
3	1	0	11110000	00000000
4	0	1	01111000	10000000
5	0	1	00111100	11000000
6	0	0	00011110	01100000

Working Set Page Replacement Algorithmus

- Die Menge an Pages, die ein Prozess zu einem bestimmten Zeitpunkt verwendet, wird als Working Set bezeichnet
 - Ist eine Teilmenge aller Pages, die ein Prozess über seine Lebensdauer verwendet
 - Prozesse verwenden meist nur einen kleinen Teil der theoretisch benötigten Pages
 - Working Set ändert sich über die Lebensdauer des Prozesses
 - Sind alle theoretisch benötigten Pages im Speicher, gibt es keine Seitenfehler
 - Trashing: Muss zwischen Instruktion eingelagert werden, weil Pages nicht im Speicher vorhanden sind, dauert das Einlagern länger als das tatsächliche Ausführen der Instruktion → schlecht für Performance
- Idee
 - OS merkt sich das Working Set eines Prozesses
 - Stellt sicher, dass das Working Set eingelagert ist, bevor ein Prozess durch den Scheduler fortgesetzt wird

Working Set Page Replacement Algorithmus

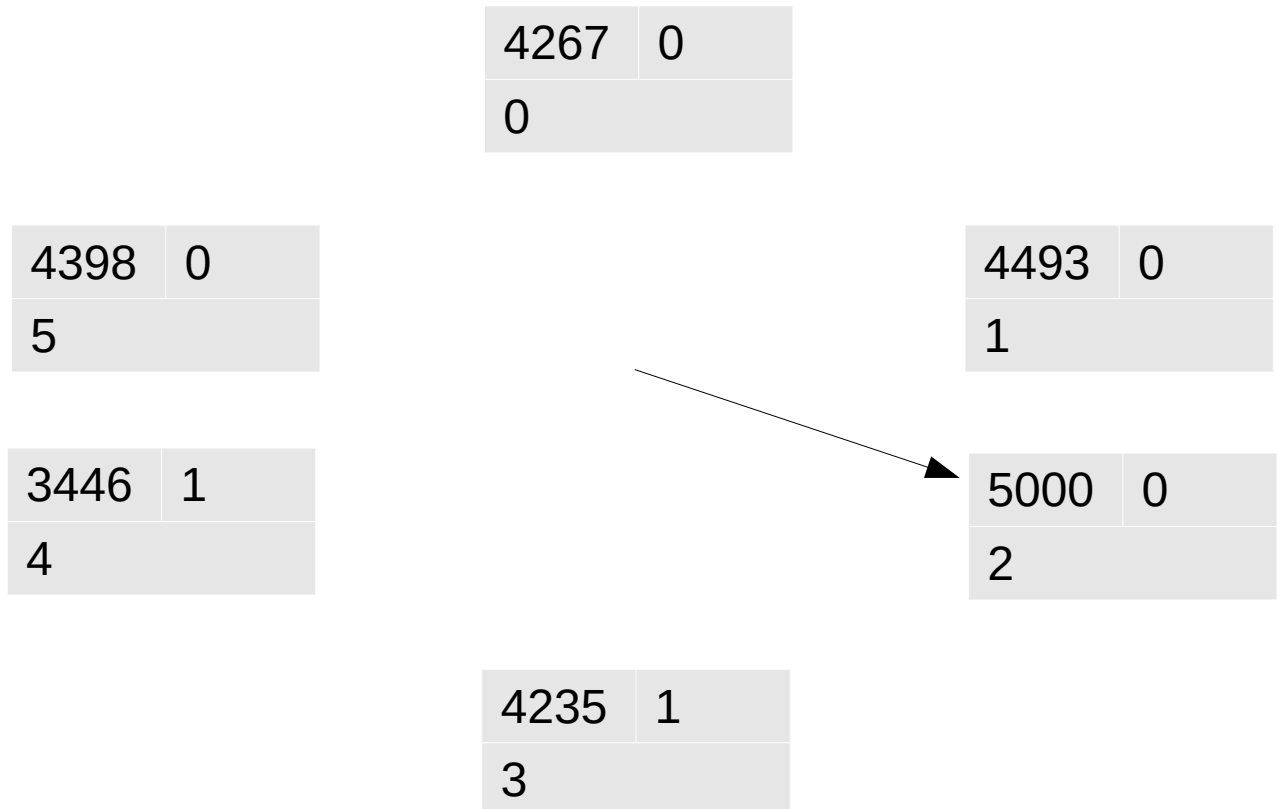
- Working Set: Menge an Pages, die in den letzten k Speicherzugriffen benutzt wurden
 - Protokollieren dieser Information würde Performance negativ beeinflussen
- Daher Vereinfachung: Menge an Pages, die in den letzten t Zeiteinheiten benutzt wurden
- Beim Ersetzen von Pages werden immer jene ersetzt, die nicht zum Working Set zählen
- Es wird zusätzlich zu den R/M Bits der Zeitpunkt des letzten Zugriffs (Alter) gespeichert
- Wir nehmen auch wieder an, dass das R Bit periodisch zurückgesetzt wird
- Seitenauslagerung wenn
 - $R == 0$ ist und
 - $\text{Alter} > t$
- Wenn $R == 0$ und $\text{Alter} \leq t$ wird Alter mit höchstem Alter ersetzt
- Tabelle wird immer komplett durchlaufen und das Alter aller Einträge aktualisiert

WSClock

- Problem des Working Set Page Replacement Algorithmus ist das Durchlaufen und Aktualisieren der kompletten Page Table bei einem Seitenfehler
- Daher: Kombination des Clock Algorithmus mit dem Working Set Page Replacement Algorithmus
- Datenstruktur ist eine ringförmige Liste (→ „Clock“)
 - Pages werden diesem Ring, wenn benötigt, hinzugefügt
 - Zusätzlich enthalten die Einträge den Zeitstempel der letzten Verwendung (und R / M Bit)
 - Es gibt einen Zeiger, der immer auf die älteste Page zeigt
 - Bei ihr wird bei einem Page Fault mit der Suche nach zu ersetzenden Pages begonnen
 - Ist $R == 0 \ \&\& \ M == 0 \ \&\& \ \text{Alter} > t$ wird
 - die Page entfernt
 - die neue Page an diese Stelle geladen
 - Der Zeiger weiterbewegt
 - Ist $R == 0 \ \&\& \ M == 1$ wird
 - Aus Performance Gründen das Schreiben der Page eingeplant
 - Der Zeiger aber weiterbewegt und die Suche fortgesetzt
 - Ist $R == 1$, wird R auf 0 gesetzt und der Zeiger weiterbewegt

WSClock - Beispiel

- Aktuelle Zeit: 5100
- t: 200
- Zeiger steht auf Page 2
 - R Bit ok, Alter nicht ok
- Zeiger wird auf Page 3 gestellt
 - R Bit nicht ok, Alter ok
 - Reset R Bit
- Zeiger wird auf Page 4 gestellt
 - Gleich zu Page 3
- Zeiger wird auf Page 5 gestellt
 - R Bit ok, Alter ok
 - Page wird ersetzt, vorausgesetzt $M == 0$

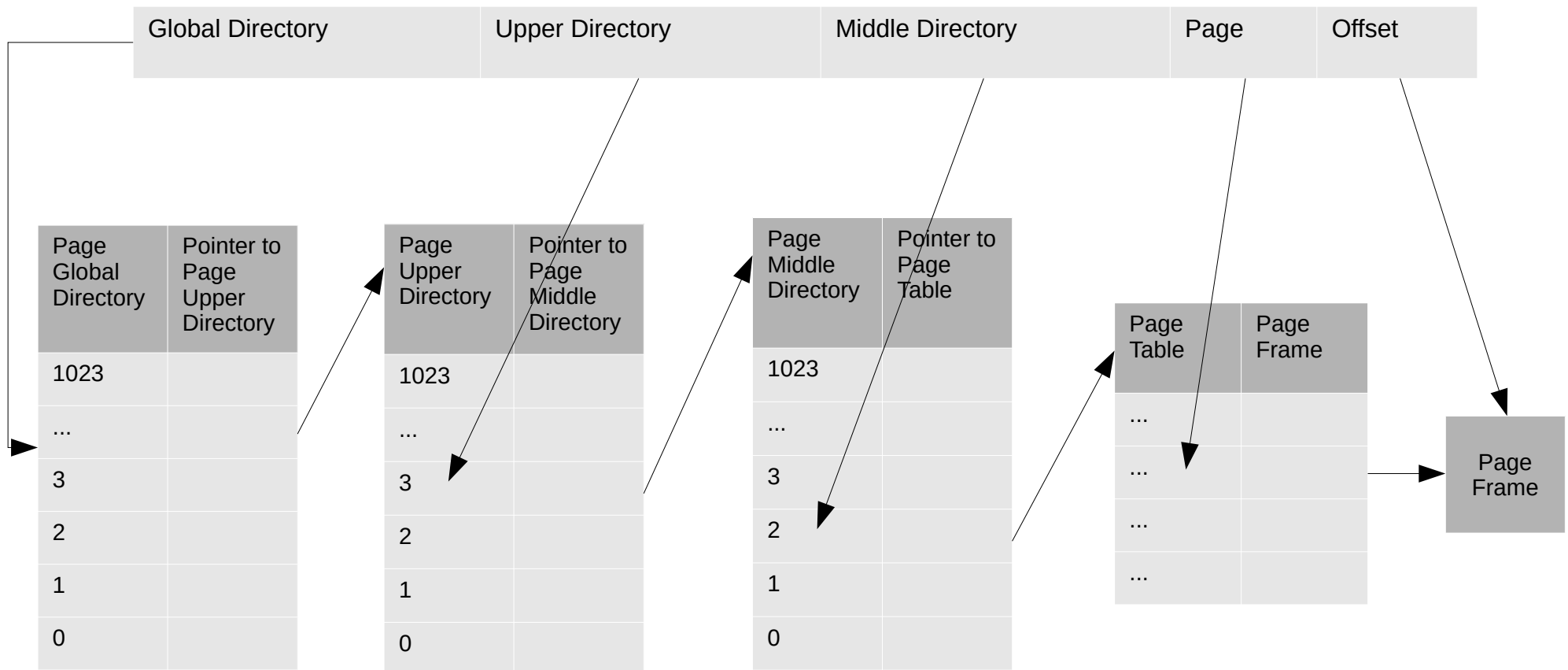


Implementierung - Linux

- Linux verwaltet Speicher mit einem Array bestehenden aus Page Deskriptoren (32 Byte → benötigt < 1% des Speichers)
 - Mapping Page → Page Frame
 - Enthält Zeiger zum Adressraum
 - Zeiger zu anderen Deskriptoren → Verlinkte Liste für z.B. freie Blöcke
- Speicher ist in Zonen unterteilt
 - Werden in einer Liste verwaltet (DMA, Normal, HighMem)
 - Enthält Informationen
 - Über Belegung, freie Bereiche
 - Active/Inactive Pages
 - Watermarks für Page Replacement Algorithmus
- Zusätzlich wird der Speicher in Nodes unterteilt
 - Für NUMA (Non-Uniform Memory Access) Systeme
 - Bei mehreren CPUs kann der Zugriff auf einen bestimmten Speicher je nach CPU unterschiedlich lange dauern
 - Muss beim Allokieren von Speicher beachtet werden
 - Es sollte nach Möglichkeit der schnellste Speicher der CPU, an die ein Prozess (CPU Affinity) gebunden ist, ausgewählt werden

Linux – Multilevel Page Tables

- Linux verwendet Multilevel Page Tables mit 4 Ebenen (Levels)
- Update auf 5 Level für 64 Bit Systeme möglich



Linux - Allocators

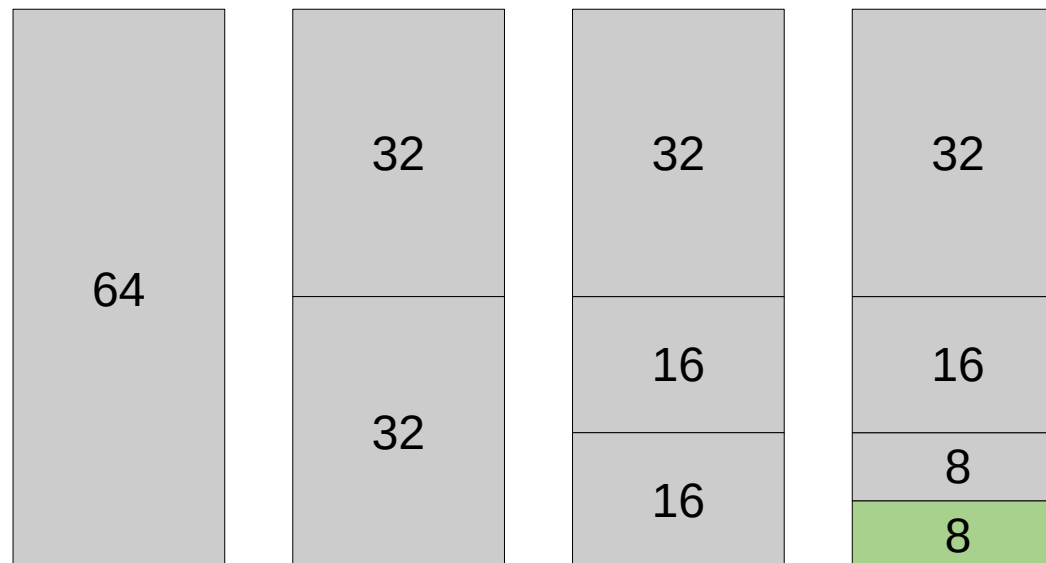
- Bis jetzt haben wir komplett ignoriert, welche Strategie ein OS bei der Zuteilung (Allokierung) von Arbeitsspeicher zu Prozessen verfolgen kann
 - Neu gestartete Prozesse müssen in den Speicher geladen werden ($\rightarrow$ `fork()`)
 - Prozesse benötigen über ihre Lebensdauer unterschiedlich große Blöcke an Speicher ($\rightarrow$ `malloc()`)
 - Prozesse geben unterschiedliche große Blöcke an Speicher über ihre Lebensdauer frei ($\rightarrow$ `free()`)
- Linux verwendet einen leicht modifizierten Buddy Algorithmus in Kombination mit Slab Allocation

Exkurs: Allokierung und Fragmentierung

- Nehmen wir an, wir stellen Speicher in 4 KB Blöcken zur Verfügung
 - Was wenn wir nur 4 Byte allokalieren?
 - Verschwendung → Interne Fragmentierung
 - Was wenn wir 2 MB allokalieren
 - Dann benötigen wir 512 4 KB Blöcke
 - Nehmen wir an, dass es sich um das Working Set eines Prozesses handelt und der TLB 64 Adressen speichern kann → Performance Problem → TLB Trashing
 - Über die Zeit wird Speicher freigegeben, dadurch entstehen Lücken und eine Durchmischung aus belegtem und freiem Speicher
 - z.B. sind noch viele kleine, nicht durchgängige Blöcke an Speicher frei, können diese aber bei einer Anforderung nach einem großen Block nicht mehr verwendet werden, d.h. theoretisch wäre noch Speicher verfügbar, dieser ist aber nicht mehr verwendbar → Externe Fragmentierung
 - D.h. es müsste der Speicher neu angeordnet werden und die freien Blöcke zu einem Bereich zusammengefasst werden (Compaction)
- Es sollte also möglich sein, Blöcke mit variabler Größe zu allokalieren
 - x86 CPUs und Linux unterstützen zusätzlich variable Page Größen

Linux - Buddy Algorithms

- Speicherblock, bestehend aus Pages, wird immer weiter unterteilt, bis die benötigte Blockgröße erreicht ist
- Bei Speicherfreigaben werden Blöcke wieder vereint
- Erweiterung Linux: Zusätzliche Listen mit unterschiedlichen Größen (1-Page Liste, 2-Page Liste, 4-Page Liste,...)
- Beispiel:
 - 64 Page Block
 - 8-Page Block wird benötigt
 - Was passiert wenn ein 16-Page Block benötigt wird?
 - Was passiert wenn danach der 8-Page Block freigegeben wird?
 - Was passiert wenn ein 33-Page Block benötigt wird?



Linux - Slab Allocation

- Buddy Algorithmus vermeidet externe Fragmentierung (leere Speicherlücken zwischen allokiertem Speicher), leidet jedoch unter interner Fragmentierung
- Zusätzlicher Slab Allocation Algorithmus reserviert mit dem Buddy Algorithmus Blöcke, welche für Objekt Caches verwendet werden
 - 1 Slab kann mehrere Objekte des selben Typs speichern, z.B. File Deskriptoren, Prozess Deskriptoren,...
 - Wird z.B. eine Datei geöffnet, wird aus einem Slab ein bereits allokiertes (aber nicht benutzter) File Deskriptor verwendet
- Slab Allocation reduziert daher die Anzahl an Allokierungs- und Freigabe-Operationen für bestimmte Typen von Objekten

Linux – Ersetzungsstrategie

- Linux verwendet kein Preloading von Pages oder Working Sets
- Linux führt Demand Paging durch → es werden nur jene Pages geladen, die auch benötigt werden
- Linux verwaltet einen Pool an freien Pages und hält diesen auf einem vernünftigen Level
- Ein Daemon (kswapd) untersucht periodisch die Watermarks von Zonen und überprüft, ob noch genug freier Speicher in den Zonen vorhanden ist
 - Ist das nicht der Fall, wird der Page Frame Reclaiming Algorithmus ausgeführt
- Eigener Daemon (pdflush) schreibt
 - periodisch alte Pages mit gesetztem w Bit auf die Festplatte
 - Oder schreibt Pages auf die Festplatte, wenn der Arbeitsspeicher knapp wird (Watermark erreicht)

Linux - Page Frame Reclaiming Algorithm

- Jede Page besitzt 2 Bits
 - Active
 - Referenced
- 2 Listen (Ringe)
 - Active Pages
 - Inactive Pages
- Pages werden zwischen den Listen aufgrund ihrer gesetzten Bits verschoben
 - Unterschiedliche Arten von allokiertem Speicher werden unterschiedlich behandelt
 - Es wird eine Mischung aus LRU und Second Chance Clock Algorithmus verwendet
 - Referenced Bit wird bei jedem Durchlauf zurückgesetzt
- kswapd gibt zuerst inactive und unreferenced Pages frei
 - Erst wenn diese Möglichkeit ausgeschöpft ist, versucht er „schwierigere“ Pages freizugeben (z.B. Shared Pages zwischen Prozessen,...)

Zusammenfassung

- Memory Abstraktionen
- Virtual Memory
- Page Tables
- MMU & TLB
- Ersetzungsstrategien
- Linux & Allokierung